

Hyperparameter optimization

2026

Model selection: coherent inference

First level: select optimal parameters:

$$\mathbf{w} = \arg \max \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})}{p(\mathcal{D}|\mathbf{h})},$$

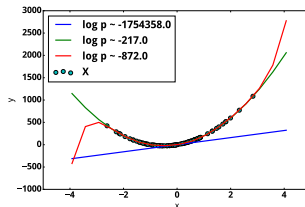
Second level: select optimal model (hyperparameters).

Evidence:

$$p(\mathcal{D}|\mathbf{h}) = \int_{\mathbf{w}} p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})d\mathbf{w}.$$



Model selection scheme



Example: polynomials

Hyperparameters

Definition

Prior for parameters $\mathbf{w}$ and structure $\mathbf{\Gamma}$ of the model $\mathbf{f}$ is a distribution $p(\mathbf{W}, \mathbf{\Gamma} | \mathbf{h}) : \mathbb{W} \times \Gamma \times \mathbb{H} \rightarrow \mathbb{R}^+$, where $\mathbb{W}$ is a parameter space, Γ is a structure space.

Definition

Hyperparameters $\mathbf{h} \in \mathbb{H}$ of the models are the parameters of $p(\mathbf{w}, \mathbf{\Gamma} | \mathbf{h})$ (parameters of prior $\mathbf{f}$).

Laplace approximation

Nonlinear case with m objects and n features: $\mathbf{y} \sim \mathcal{N}(\mathbf{f}(\mathbf{X}, \mathbf{w}), \lambda^{-1})$, $\mathbf{w} \sim \mathcal{N}(0, \mathbf{A}^{-1})$.

Write integral:

$$p(\mathcal{D}|\mathbf{h}) = p(\mathbf{y}|\mathbf{X}, \mathbf{A}, \lambda) = \frac{\sqrt{\lambda \cdot |\mathbf{A}|}}{\sqrt{(2\pi)^{m+n}}} \int_{\mathbf{w}} \exp(-S(\mathbf{w})) d\mathbf{w}.$$

Using Taylor series for S :

$$S(\mathbf{w}) \approx S(\hat{\mathbf{w}}) + \frac{1}{2} \Delta \mathbf{w}^T \mathbf{H} \Delta \mathbf{w}$$

Integral reduces to the following expression:

$$\frac{\sqrt{\lambda \cdot |\mathbf{A}|}}{\sqrt{(2\pi)^{m+n}}} S(\hat{\mathbf{w}}) \int_{\mathbf{w}} \exp(-\frac{1}{2} \Delta \mathbf{w}^T \mathbf{H} \Delta \mathbf{w}) d\mathbf{w}$$

The expression under integral corresponds to the unnormalized Gaussian PDF.

Graves, 2011

Prior: $p(\mathbf{w}|\sigma) \sim \mathcal{N}(\boldsymbol{\mu}, \sigma \mathbf{I})$.

Variational inference: $q(\mathbf{w}) \sim \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q \mathbf{I})$.

Greedy optimization:

$$\boldsymbol{\mu} = \hat{\mathbf{E}}\mathbf{w}, \quad \sigma = \hat{\mathbf{D}}\mathbf{w}.$$

Prune w_i using relative PDF:

$$\lambda = \frac{q(\mathbf{0})}{q(\boldsymbol{\mu}_{i,q})} = \exp\left(-\frac{\mu_i^2}{2\sigma_i^2}\right).$$



$\lambda = 0.01$



$\lambda = 0.05$



$\lambda = 0.1$



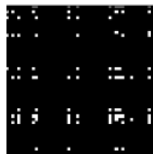
$\lambda = 0.2$



$\lambda = 0.5$



$\lambda = 1$



$\lambda = 2$

Problem statement

Let $\boldsymbol{\theta} \in \mathbb{R}^s$ be the set of all the optimized parameters (including variational parameters if needed).

$L(\boldsymbol{\theta}, \mathbf{h})$ is a differential loss function $\mathbf{f}$.

$Q(\boldsymbol{\theta}, \mathbf{h})$ is a differential validation function.

The problem is to find optimal parameters $\boldsymbol{\theta}^*$ and hyperparameters $\mathbf{h}^*$ of the model that minimize

$$\mathbf{h}^* = \arg \max_{\mathbf{h} \in \mathbb{H}} Q(\boldsymbol{\theta}^*(\mathbf{h}), \mathbf{h}),$$

$$\boldsymbol{\theta}(\mathbf{h})^* = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^s} L(\boldsymbol{\theta}, \mathbf{h}).$$

Bayesian inference

Let $\theta = [\mathbf{w}]^\top$.

First level:

$$\theta^* = \arg \max(-L(\theta, \mathbf{h})) = p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \mathbf{h}) = \frac{p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w}|\mathbf{h})}{p(\mathbf{y}|\mathbf{X}, \mathbf{h})}.$$

Second level:

$$p(\mathbf{h}|\mathbf{X}, \mathbf{y}) \propto p(\mathbf{y}|\mathbf{X}, \mathbf{h})p(\mathbf{h}),$$

Considering $p(\mathbf{h})$ improper flat prior we get the following expression:

$$Q(\theta, \mathbf{h}) = p(\mathbf{y}|\mathbf{X}, \mathbf{h}) = \int_{\mathbf{w} \in \mathbb{R}^u} p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w}|\mathbf{h}) \rightarrow \max_{\mathbf{h} \in \mathbb{H}}.$$

Cross-validation

Split the dataset $\mathfrak{D}$ into k equal (maybe stratified) parts:

$$\mathfrak{D} = \mathfrak{D}_1 \sqcup \dots \sqcup \mathfrak{D}_k.$$

Optimize k models for each data part. Let $\theta = [\mathbf{w}_1, \dots, \mathbf{w}_k]$, where $\mathbf{w}_1, \dots, \mathbf{w}_k$ are the model parameters for optimization k .

Let L be a loss function:

$$L(\theta, \mathbf{h}) = -\frac{1}{k} \sum_{q=1}^k \left(\frac{k}{k-1} \log p(\mathbf{y} \setminus \mathbf{y}_q | \mathbf{X} \setminus \mathbf{X}_q, \mathbf{w}_q) + \log p(\mathbf{w}_q | \mathbf{h}) \right). \quad (1)$$

Let Q be a validation loss:

$$Q(\theta, \mathbf{h}) = \frac{1}{k} \sum_{q=1}^k k \log p(\mathbf{y}_q | \mathbf{X}_q, \mathbf{w}_q).$$

ELBO

Let $L = -Q$:

$$\log p(\mathbf{y}|\mathbf{X}, \mathbf{A}) \geq \sum_{\mathbf{x}, y} \log p(y|\mathbf{x}, \hat{\mathbf{w}}) - D_{\text{KL}}(q(\mathbf{w})||p(\mathbf{w}|\mathbf{A})) = -L(\theta, \mathbf{A}^{-1}) = Q(\theta, \mathbf{A}^{-1}),$$

where q is a normal distribution with diagonal covariance matrix:

$$q \sim \mathcal{N}(\mu_q, \mathbf{A}_q^{-1}),$$

$$D_{\text{KL}}(q(\mathbf{w})||p(\mathbf{w}|\mathbf{f})) = \frac{1}{2}(\text{Tr}[\mathbf{A}\mathbf{A}_q^{-1}] + (\mu - \mu_q)^{\top} \mathbf{A}(\mu - \mu_q) - u + \ln |\mathbf{A}^{-1}| - \ln |\mathbf{A}_q^{-1}|).$$

Use variational parameters of q as a vector of optimized parameters θ :

$$\theta = [\alpha_1, \dots, \alpha_u, \mu_1, \dots, \mu_u].$$

Evidence vs CV

Evidence estimation:

$$\log p(\mathcal{D}|\mathbf{f}) = \log p(\mathcal{D}_1|\mathbf{f}) + \log p(\mathcal{D}_2|\mathcal{D}_1, \mathbf{f}) + \cdots + \log p(\mathcal{D}_n|\mathcal{D}_1, \dots, \mathcal{D}_{n-1}, \mathbf{f}).$$

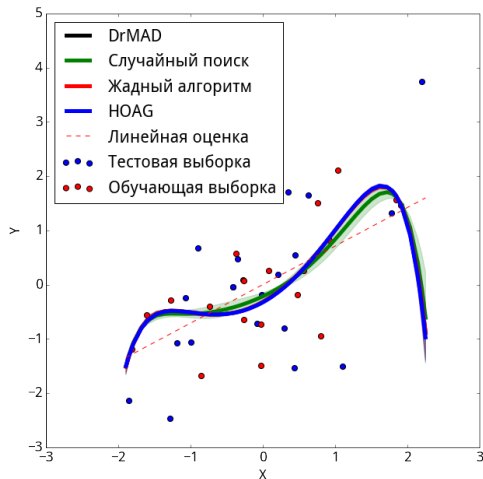
Leave-one-out estimation:

$$\text{LOU} = \mathbb{E} \log p(\mathcal{D}_n|\mathcal{D}_1, \dots, \mathcal{D}_{n-1}, \mathbf{f}).$$

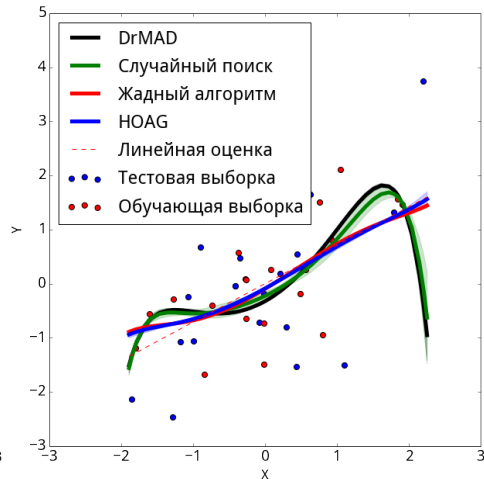
Cross-validation uses expected values of the last term $p(\mathcal{D}_n|\mathcal{D}_1, \dots, \mathcal{D}_{n-1}, \mathbf{f})$ as a complexity estimation.

Evidence considers **full** complexity.

Experiment: polynoms



CV



Evidence

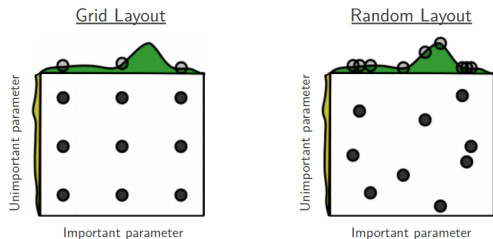
Basic methods of hyperparameter optimization

Variants:

- Grid search;
- random search.

Both methods suffer from curse of dimensionality.¹

The random search can be more effective if the hyperparameter space is degenerate.



Bergstra et al., 2012

¹see the talk of Nikita Kiselev on the random projection optimization for this problem.

Sequential model-based optimization (SMBO)

Algorithm Framework 1: Sequential Model-Based Optimization (SMBO)

R keeps track of all target algorithm runs performed so far and their performances (*i.e.*, SMBO's training data $\{([\theta_1, \mathbf{x}_1], o_1), \dots, ([\theta_n, \mathbf{x}_n], o_n)\}$), $\mathcal{M}$ is SMBO's model, $\vec{\Theta}_{new}$ is a list of promising configurations, and t_{fit} and t_{select} are the runtimes required to fit the model and select configurations, respectively.

Input : Target algorithm A with parameter configuration space Θ ; instance set Π ; cost metric $\hat{c}$

Output : Optimized (incumbent) parameter configuration, θ_{inc}

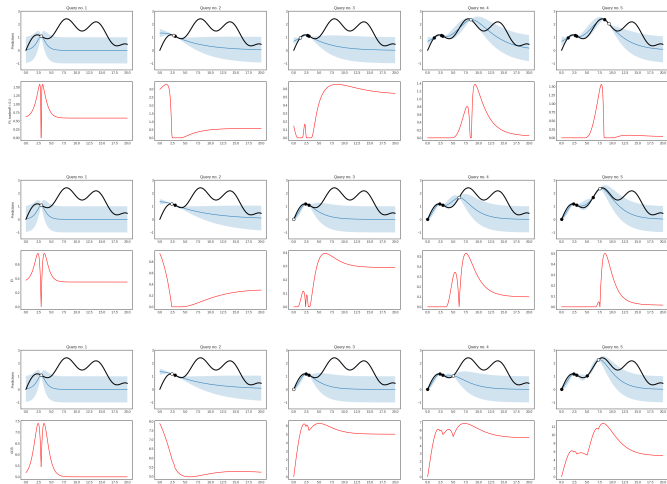
```
1  $[\mathbf{R}, \theta_{inc}] \leftarrow \text{Initialize}(\Theta, \Pi)$ 
2 repeat
3    $[\mathcal{M}, t_{fit}] \leftarrow \text{FitModel}(\mathbf{R})$ 
4    $[\vec{\Theta}_{new}, t_{select}] \leftarrow \text{SelectConfigurations}(\mathcal{M}, \theta_{inc}, \Theta)$ 
5    $[\mathbf{R}, \theta_{inc}] \leftarrow \text{Intensify}(\vec{\Theta}_{new}, \theta_{inc}, \mathcal{M}, \mathbf{R}, t_{fit} + t_{select}, \Pi, \hat{c})$ 
6 until total time budget for configuration exhausted
7 return  $\theta_{inc}$ 
```

Next point to estimate

Next point selection is done using *Acquisition function*:

- Upper confidence level
- Probability of Improvement: $P(I(\mathbf{h}) > 0)$, $I(\mathbf{h}) = \max(L(\mathbf{h}) - L(\mathbf{h}^*), 0)$
- Expected improvement $EI(\mathbf{h})$

Acquisition functions



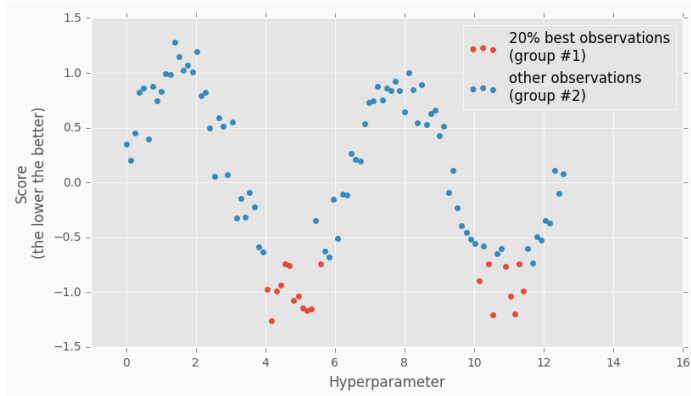
From modAL python library: PI, EI, UCB.

Tree Parzen estimator

Basic idea

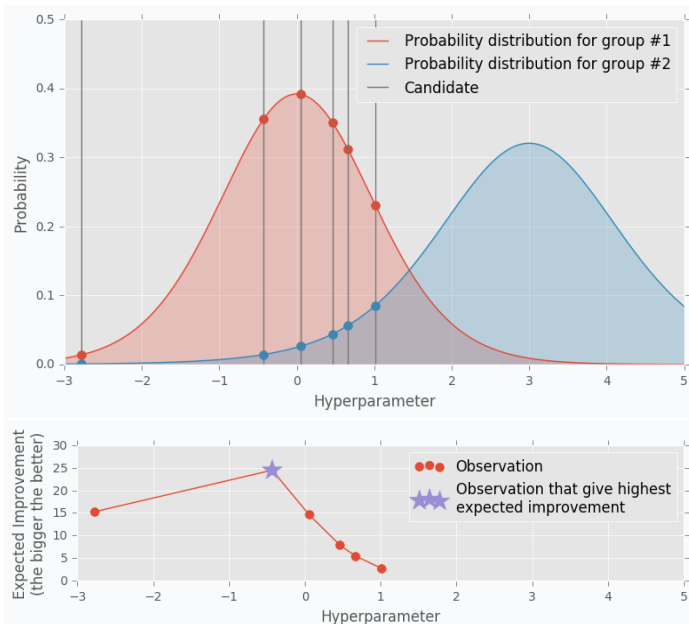
- Sample multiple hyperparameter instances $\mathbf{h}_i$
- Fit models $\mathbf{f}_i$ using $\mathbf{h}_i$
- Select models from λ -quantile of model results Loss_λ and fit adaptive Parzen estimator p_1
- Select remaining models and fit adaptive Parzen estimator p_2
- Sample new hyperparameter $\mathbf{h}$ that maximizes Expected improvement: $EI(\mathbf{h})$.

TPE



From NeurPy library.

TPE



Gaussian process, definition (wiki)

- A random process f_t with continuous time is gaussian if and only if for each finite set of indices $t_1, \dots, t_k$: $f_{t_1}, \dots, f_{t_k}$ is a multivariate Gaussian variable.
- Each linear combination $f_{t_1}, \dots, f_{t_k}$ is a univariate Gaussian.

Definition (simplified)

Define a Gaussian process $\mathcal{GP}(\mathbf{m}(\mathbf{x}), \mathbf{k}(\mathbf{x}, \mathbf{x}'))$ to be a distribution on the set of functions that for each $\mathbf{x}, \mathbf{x}'$: $\mathcal{GP}(\mathbf{m}(\mathbf{x}), \mathbf{k}(\mathbf{x}, \mathbf{x}'))$ is a Gaussian distribution.

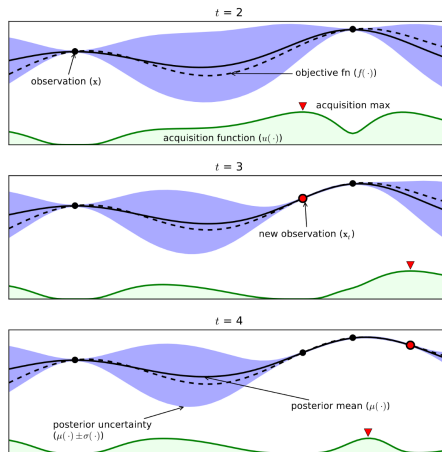
Gaussian process

Idea: Model $Q(\theta(\mathbf{h})^*, \mathbf{h})$ using Gaussian process depending on $\mathbf{h}$.

Pros:

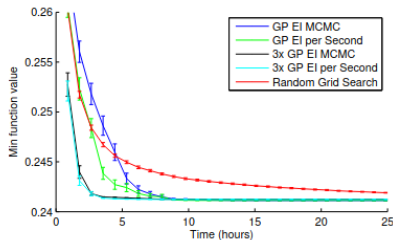
- Flexibility.
- Probabilistic model, cheaper than exhaustive search.

Cons: cubic complexity on the sample number.

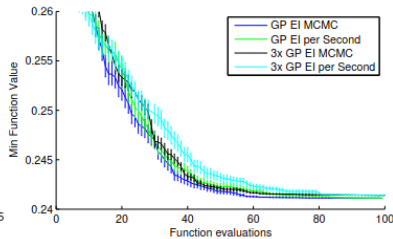


Shahriari et. al, 2016. GP example.

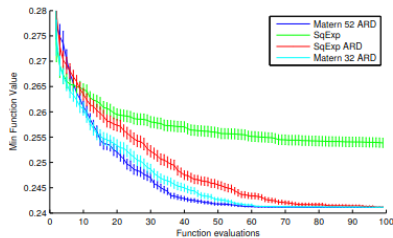
Hyperparameter optimization



(a)

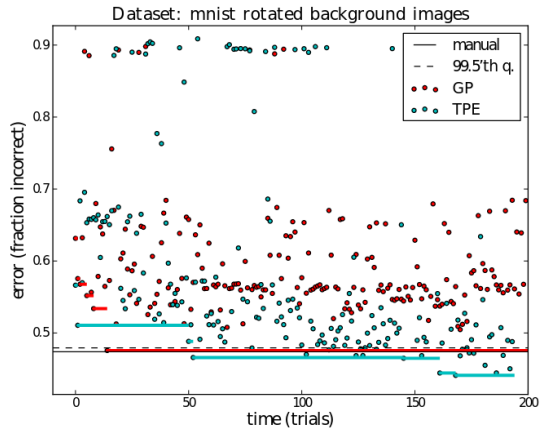
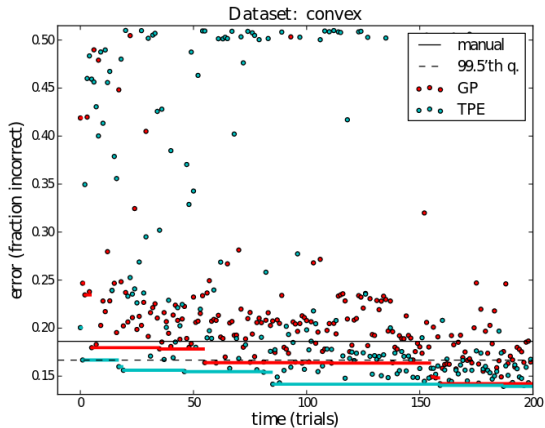


(b)

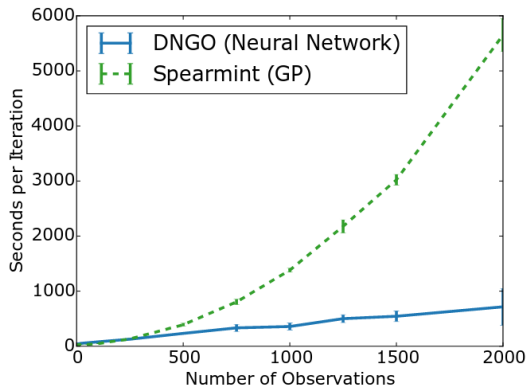


(c)

TPE vs GP



GP: complexity challenge



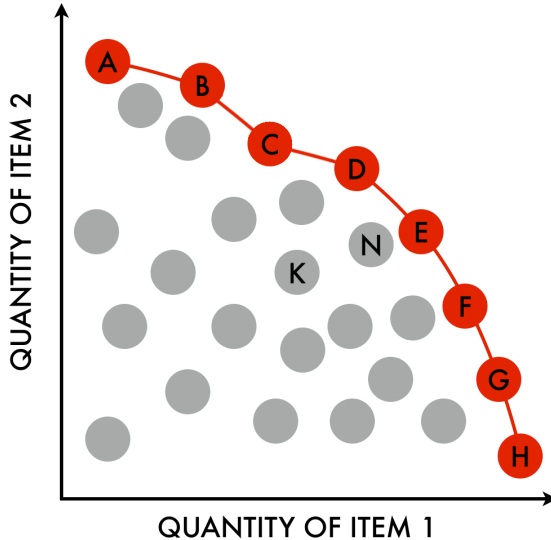
(Snoek, 2015)

Muilti-objective optimization

Can we use multiple criteria for optimization task?

Muilti-objective optimization

Can we use multiple criteria for optimization task?



References

- Bishop C. M. Pattern recognition //Machine learning. – 2006. – T. 128. – №. 9.
- Bakhteev O. Y., Strijov V. V. Comprehensive analysis of gradient-based hyperparameter optimization algorithms //Annals of Operations Research. – 2020. – T. 289. – №. 1. – C. 51-65.
- Graves A. Practical variational inference for neural networks //Advances in neural information processing systems. – 2011. – T. 24.
- Bergstra et al., Random Search for Hyper-Parameter Optimization, 2012
- Bobak Shahriari et. al, Taking the Human Out of the Loop: A Review of Bayesian Optimization, 2016
- Daniel McDuff, Gaussian Processes,
<https://courses.media.mit.edu/2010fall/mas622j/ProblemSets/slidesGP.pdf>
- Chris Williams, Gaussian Processes for Machine Learning,
<https://www.newton.ac.uk/files/seminar/20070809140015001-150844.pdf>
- Ed Snelson, utorial: Gaussian process models for machine learning,
<https://mlg.eng.cam.ac.uk/tutorials/06/es.pdf>
- Snoek J., Larochelle H., Adams R. P. Practical bayesian optimization of machine learning algorithms //Advances in neural information processing systems. – 2012. – T. 25.
- Snoek J. et al. Scalable bayesian optimization using deep neural networks //International conference on machine learning. – PMLR, 2015. – C. 2171-2180.
- modAL: A modular active learning framework for Python3
- NeurPY

Organizational issues

- Classes time: when?
- Scoring system: will be slightly adjusted
 - ▶ Number of talks (not sure)
 - ▶ More fair scores on the project
 - ▶ PoC vs Demo
 - ▶ Remove '+1' bonus score

Next hometask: project and roles decision

Projects

Bensemble: TBA

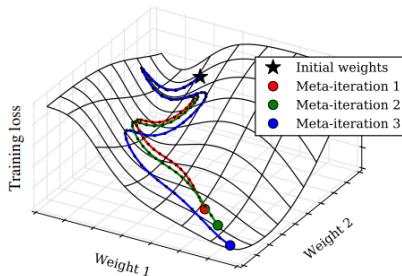
Gradient methods

Idea: Optimize hyperparameters using the full parameter optimization trajectory.

Pros:

- Hyperparameter optimization will consider the features of the parameter optimization.
- The complexity is linear on the number of hyperparameters.

Cons: the computational cost is very expensive. Also can be problems with numerical precision of reverse method.



Maclaurin et. al, 2015. Example.

Project: Gradient-based hyperparameter optimization

Hyperparameter optimization is a problem of finding suitable hyperparameters given a validation (or sometimes test) dataset. In contrast to classical hyperparameter optimization methods, gradient-based methods allow the researchers to perform hyperparameter optimization over a billion-dimension search space.

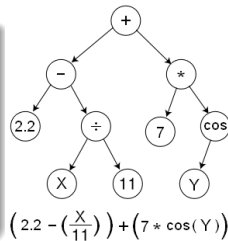
Recommended stack: JAX or pytorch. As with Evidence-based operators, the main problem here is to make this usable for a broad-class of tasks (provide good API).

The code of 3 algorithms is already implemented on JAX by Konstantin Yakovlev.
There are 2 projects for this topic: on short and long hoptimization horizon

Symbolic models: recap

Definition, Koza

The problem of **symbolic function identification** is developing a composition of terminals and functions that can return the correct value of the function after seeing a finite sampling of combinations of the independent variable associated with the correct value of the dependent variable



Definition, wikipedia

Symbolic regression is a type of regression analysis that searches the space of mathematical expressions to find the model that best fits a given dataset, both in terms of accuracy and simplicity.

Expression tree as it can be used in symbolic regression to represent a function

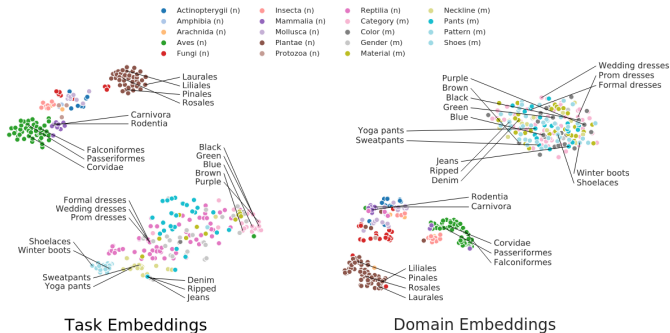
Project: Neural symbolic regression

symbolic regression is a promising class of machine learning/data analysis methods that try to restore the original model treating the model in a symbolic way. The most well known symbolic regression models are implemented using evolutionary algorithms. The other methods, including Bayesian optimization are usually implemented only as a proof-of-concept. In this project we focus on the implementation of neural network-based methods for symbolic regression.

First algorithm to implement: is a vanilla symbolic regression with evolutionary algorithms.

Project: Dataset / Task embeddings

One of the developing concepts in contemporary meta-learning is task and dataset embedding. The ability to embed tasks or datasets into a common vector space enables the study of dataset similarity, model transferability, and task relationships.



Task2vec